

React Memo

React.memo is a **Higher-Order Component (HOC)** that **prevents unnecessary re-renders** of a functional component.

React.memo is a Higher-Order Component that performs shallow comparison of props and skips re-rendering if props have not changed. It helps optimize performance in components that render frequently with stable props.

```
const MemoizedComponent = React.memo(MyComponent)
```

It tells React:

“Only re-render this component if its props have changed.”

```
function Parent() {  
  const [count, setCount] = useState(0)  
  
  return (  
    <>  
    <button onClick={() => setCount(count + 1)}>+</button>  
    <Child name="Prajwal" />  
    </>  
  )  
}
```

By default:

- 👉 When a parent re-renders
- 👉 All child components re-render
- 👉 Even if their props didn't change

How React.memo Works Internally

When you wrap a component:

```
const Child = React.memo(function Child({ name }) {  
  console.log("Child rendered")  
  return <div>{name}</div>  
})
```

Stores previous props

On next render:

- Compares old props vs new props
- Uses **shallow comparison**
- If equal → skips rendering
- If different → re-renders

This reduces:

- Function execution
- Virtual DOM diffing
- Child reconciliation

Shallow Comparison

React.memo uses **shallow comparison**.

It checks:

```
prevProps === nextProps
```

For primitives:

```
name = "Prajwal" // works fine
```

For objects/arrays/functions:

```
{ user: { name: "Prajwal" } }
```

✗ Even if contents same

✓ If reference changes → re-render happens

```
<Child user={{ name: "Prajwal" }} />
```

Every render creates a new object → reference changes → memo fails.

Fixing That: useMemo and useCallback

If you pass:

- Objects → use useMemo
- Functions → use useCallback

Now references stay stable.

Custom Comparison Function

You can provide your own equality check:

```
const Child = React.memo(  
  function Child({ user }) {  
    return <div>{user.name}</div>  
  },  
  (prevProps, nextProps) => {  
    return prevProps.user.name === nextProps.user.name  
  }  
)
```

If function returns:

- true → skip render
- false → re-render

⚠ Be careful:

- If comparison is heavy → you may hurt performance
- Comparison itself runs every parent render

When React.memo Actually Helps

- ✓ Large lists (like 1000 rows)
- ✓ Expensive components (charts, maps, animations)

- ✓ Stable props
- ✓ Frequently re-rendering parent

Use React.memo when:

- ✓ Component is pure
- ✓ Props are stable
- ✓ Render is expensive
- ✓ Parent re-renders often

When It Does NOT Help

- ✗ Small components
- ✗ Props always changing
- ✗ Parent rarely re-renders
- ✗ You don't measure performance

Remember:

React.memo adds comparison cost.

If rendering is cheap, comparison might be more expensive than rendering.

Rendering cost vs Comparison cost

If:

Comparison cost < Rendering cost

→ Use memo

If:

Comparison cost > Rendering cost

→ Don't use memo

```
const UserCard = React.memo(({ user }) => {
  console.log("UserCard rendered");
  return <div>{user.name}</div>;
});

export default function App() {
  const [count, setCount] = React.useState(0);

  const user = { name: "Prajwal" };

  return (
    <>
      <button onClick={() => setCount(count + 1)}>Increment</button>
      <UserCard user={user} />
    </>
  );
}
```

Step 1 — Why is UserCard re-rendering?

You tell me first.

But here's how a senior thinks:

React.memo does:

Shallow comparison of previous props vs next props.

That means:

```
prevProps.user === nextProps.user
```

Now look at this line:

```
const user = { name: "Prajwal" };
```

Every render of App:

- A **new object** is created
- New reference in memory

- Even though contents are same

So:

```
{ name: "Prajwal" } !== { name: "Prajwal" }
```

Different references → React thinks props changed → re-render.

Step 2 — Fix It Properly

Correct Fix

```
const user = React.useMemo(() => ({ name: "Prajwal" }), []);
```

Now:

- Object is created once
- Same reference reused
- Shallow comparison passes
- UserCard does NOT re-render

Full corrected version:

```
export default function App() {  
  const [count, setCount] = React.useState(0);  
  
  const user = React.useMemo(() => ({ name: "Prajwal" }), []);  
  
  return (  
    <>  
      <button onClick={() => setCount(count + 1)}>Increment</button>  
      <UserCard user={user} />  
    </>  
  );  
}
```

Step 3 — Performance Tradeoff

Now this is where most devs stop thinking.

Let's analyze.

Before fix:

- Component re-renders
- But rendering `<div>{user.name}</div>` is cheap

After fix:

- We add:
 - useMemo tracking
 - Dependency array comparison
 - Extra memory retention

If UserCard is tiny → you might have made performance worse.

Memoization has cost.